React Development — From JavaScript to Component-Based Applications

Prepared By: Yared Kebede

Below is a **practice-first React course module** designed for students who have already completed HTML, CSS, and JavaScript. The course deliberately starts with the problems of traditional frontend development, then introduces modern JavaScript tooling and finally moves into React.

Module 1 — Why React?

Learning Objectives

By the end of this module, students should be able to:

- Explain the limitations of a large HTML/CSS/JavaScript application.
- Understand why reusable components are useful.
- Understand ES Modules.
- Explain the purpose of `npm` and `package.json`.
- Understand what a build tool does.
- Create a React project using Vite.
- Explain the basic React mental model.

Part 1 — Remember What We Already Know

Before React, we can build websites using:

```
HTML
↓
Structure

CSS
↓
Appearance

JavaScript
↓
Behavior
```

For example:

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Dashboard</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <h1>Student Dashboard</h1>

    <button id="addStudent">
        Add Student
    </button>

    <div id="students"></div>

    <script src="app.js"></script>
</body>
</html>
```

style.css

```
.student-card {
    border: 1px solid #ddd;
    padding: 20px;
    margin: 10px 0;
    border-radius: 8px;
}
```

app.js

```
const students = [
    {
        name: "Abebe",
        course: "JavaScript"
    },
    {
        name: "Hana",
        course: "HTML & CSS"
    }
];

const container = document.querySelector("#students");

students.forEach(student => {
    const card = document.createElement("div");

    card.className = "student-card";

    card.innerHTML = `
        <h2>${student.name}</h2>
        <p>${student.course}</p>
    `;

    container.appendChild(card);
});
```

This works.

And this is important:

There is nothing wrong with HTML, CSS and JavaScript.

The problem appears when the application becomes **large and complicated**.

Part 2 — What Happens When the Application Gets Bigger?

Imagine that our student dashboard grows.

We now need:

```
Dashboard
├── Navbar
├── Sidebar
├── Student List
├── Student Card
├── Course List
├── Course Card
├── Notifications
├── Profile
├── Settings
├── Search
├── Filters
├── Modals
└── Forms
```

And our JavaScript contains:

```
DOM manipulation
Event handlers
API calls
Validation
UI rendering
State
Business logic
Authentication
Filtering
Searching
```

Eventually:

```
app.js
↓
500 lines
↓
1000 lines
↓
```

```
3000 lines
```

The problem isn't that JavaScript can't handle this.

The problem is **organization**.

Part 3 — The First Problem: Repetition

Imagine displaying three students.

```
<div class="student-card">
    <h2>Abebe</h2>
    <p>JavaScript</p>
</div>

<div class="student-card">
    <h2>Hana</h2>
    <p>React</p>
</div>

<div class="student-card">
    <h2>Samuel</h2>
    <p>Python</p>
</div>
```

What if we have 100 students?

We don't want to manually write 100 cards.

So JavaScript can help.

```
function createStudentCard(student) {
    return `
        <div class="student-card">
            <h2>${student.name}</h2>
            <p>${student.course}</p>
        </div>
    `;
}
```

Then:

```
students.forEach(student => {
    container.innerHTML += createStudentCard(student);
});
```

We have created something reusable.

Important concept

This function represents a **piece of UI that can be reused**.

That idea will eventually become a React component.

Part 4 — The Second Problem: One JavaScript File

Suppose our application now contains:

```
// students
// courses
// authentication
// navigation
// modal
// API
// validation
// notifications
// dashboard
// settings
```

Everything is inside:

```
app.js
```

We don't want that.

We want:

```
students.js
courses.js
auth.js
api.js
ui.js
dashboard.js
```

But now another question appears:

How can these JavaScript files communicate with each other?

This leads us to **ES Modules**.

Part 5 — ES Modules

ES Modules allow us to split JavaScript into separate files.

Create:

```
project/
```

```
├── index.html
└── js/
    ├── students.js
    ├── courses.js
    └── app.js
```

students.js

```
export const students = [
    {
        name: "Abebe",
        course: "React"
    },
    {
        name: "Hana",
        course: "JavaScript"
    }
];
```

The `export` keyword makes the variable available to other modules.

app.js

```
import { students } from "./students.js";

console.log(students);
```

And in HTML:

```
<script type="module" src="./js/app.js"></script>
```

Exporting Functions

We can also export functions.

calculations.js

```
export function calculateAverage(scores) {
    const total = scores.reduce((sum, score) => sum + score, 0);

    return total / scores.length;
}
```

app.js

```
import { calculateAverage } from "./calculations.js";

const average = calculateAverage([80, 90, 70]);

console.log(average);
```

Now our application can be organized into modules.

Why Modules Matter

Without modules:

```
One huge JavaScript file
```

With modules:

```
students.js
courses.js
auth.js
api.js
ui.js
```

This gives us:

- organization
- reusability
- separation of responsibilities
- easier maintenance

But modern applications need more than modules.

Part 6 — Where Do External Packages Come From?

Imagine we want to use a library.

For example:

```
import something from "some-library";
```

Where does `"some-library"` come from?

We need a way to:

- install packages
- track dependencies
- manage versions
- run development commands
- build applications

This is where **npm** enters.

Part 7 — npm

`npm` is a package manager for JavaScript projects.

Create a project:

```
npm init -y
```

This creates:

```
package.json
```

A basic `package.json` might look like:

```
{
    "name": "student-dashboard",
    "version": "1.0.0",
    "description": "Student dashboard",
    "scripts": {},
    "dependencies": {}
}
```

Part 8 — Installing a Package

Let's install a package:

```
npm install lodash
```

Now:

```
package.json
package-lock.json
node_modules/
```

The dependency is recorded in:

```
{
    "dependencies": {
        "lodash": "^4.17.21"
    }
}
```

We can then use it:

```
import _ from "lodash";

const numbers = [1, 2, 3, 4, 5];

console.log(_.reverse(numbers));
```

The important idea is:

```
package.json
        ↓
Dependencies
        ↓
npm
        ↓
node_modules
```

Part 9 — Why Do We Need a Build Tool?

Students can now ask:

If JavaScript already supports modules, why do React applications need tools like Vite?

Because modern applications need more than simply loading JavaScript files.

A development tool can help with:

- bundling modules
- transforming JSX
- development server
- fast updates
- optimization
- production builds
- handling assets
- environment configuration

We don't need to teach every internal detail yet.

The important mental model is:

```
Our source code
      ↓
Build tool
      ↓
Browser-ready application
```

Part 10 — Creating Our React Project

Now students are ready for React.

We will use **Vite**.

```
npm create vite@latest
```

Choose:

```
React
JavaScript
```

Then:

```
cd project-name
npm install
npm run dev
```

The browser will show our React application.

Part 11 — Understanding the React Project

A typical project contains:

```
student-dashboard/
│
├── node_modules/
├── public/
│
├── src/
│   ├── assets/
│   ├── App.jsx
│   └── main.jsx
│
├── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

Don't skip this part.

Students should understand what they are looking at.

---

package.json

Contains project information, scripts and dependencies.

Example:

```
{
    "scripts": {
        "dev": "vite",
        "build": "vite",
        "preview": "vite"
    }
}
```

So:

```
npm run dev
```

means:

Run the development server.

And:

```
npm run build
```

means:

Create the production version of the application.

---

main.jsx

This is where React starts.

A simplified version:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
    <StrictMode>
        <App />
    </StrictMode>
);
```

The important relationship is:

```
index.html
     ↓
main.jsx
     ↓
App.jsx
     ↓
React application
```

Part 12 — What Is React?

Now introduce React.

React allows us to build user interfaces from **components**.

Instead of thinking:

```
HTML
CSS
```

```
JavaScript
```

we start thinking:

```
Application
│
├── Navbar
├── Sidebar
├── StudentList
│   └── StudentCard
├── CourseList
│   └── CourseCard
└── Footer
```

Each piece can be a component.

Part 13 — Our First Component

App.jsx

```
function App() {
    return (
        <h1>
            Student Dashboard
        </h1>
    );
}

export default App;
```

Notice:

```
<h1>Student Dashboard</h1>
```

looks like HTML.

But it is inside JavaScript.

This is **JSX**.

Part 14 — JSX

JSX allows us to describe UI using syntax that looks similar to HTML.

Example:

```
function App() {
    const name = "Abebe";
```

```
    return (
        <div>
            <h1>Welcome</h1>
            <p>Hello {name}</p>
        </div>
    );
}
```

The JavaScript expression:

```
{name}
```

is inserted into the UI.

Important: JSX Is Not HTML

Students should understand this early.

JSX is syntax that gets transformed into JavaScript.

Conceptually:

```
<h1>Hello</h1>
```

becomes something React can use to describe the UI.

Therefore JSX gives us the ability to combine:

```
JavaScript logic
+
UI description
```

Part 15 — Creating Components

Create:

```
function Navbar() {
    return (
        <nav>
            Student Dashboard
        </nav>
    );
}
```

Then:

```
function Sidebar() {
    return (
```

```
        <aside>
            <p>Dashboard</p>
            <p>Students</p>
            <p>Courses</p>
        </aside>
    );
}
```

Now use them:

```
function App() {
    return (
        <>
            <Navbar />
            <Sidebar />

            <main>
                <h1>Dashboard</h1>
            </main>
        </>
    );
}
```

We now have:

```
App
├── Navbar
├── Sidebar
└── Main
```

Part 16 — Components Are Reusable

Suppose we need a student card.

Create:

`StudentCard.jsx`

```
function StudentCard() {
    return (
        <div className="student-card">
            <h2>Abebe Kebede</h2>
            <p>React Development</p>
        </div>
    );
}

export default StudentCard;
```

Use it:

```
import StudentCard from "./StudentCard";

function App() {
    return (
```

```
        <div>
            <StudentCard />
            <StudentCard />
            <StudentCard />
        </div>
    );
}
```

We don't have to duplicate the markup.

Part 17 — Props

But every student shouldn't have the same information.

We need to pass data.

This is where **props** come in.

StudentCard.jsx

```
function StudentCard({ name, course }) {
    return (
        <div className="student-card">
            <h2>{name}</h2>
            <p>{course}</p>
        </div>
    );
}

export default StudentCard;
```

Then:

```
<StudentCard
    name="Abebe Kebede"
    course="React"
/>

<StudentCard
    name="Hana Alemu"
    course="JavaScript"
/>
```

The component is the same.

The data is different.

Part 18 — Rendering Arrays

Instead of manually writing:

```
<StudentCard />
<StudentCard />
<StudentCard />
```

we can use our JavaScript knowledge.

```
const students = [
    {
        id: 1,
        name: "Abebe",
        course: "React"
    },
    {
        id: 2,
        name: "Hana",
        course: "JavaScript"
    },
    {
        id: 3,
        name: "Samuel",
        course: "Python"
    }
];
```

Then:

```
{students.map(student => (
    <StudentCard
        key={student.id}
        name={student.name}
        course={student.course}
    />
))}
```

This is an excellent moment to tell students:

React didn't replace JavaScript. React uses JavaScript heavily.

Part 19 — Events

Students already know:

```
button.addEventListener("click", handleClick);
```

React uses:

```
<button onClick={handleClick}>
    Click Me
</button>
```

Example:

```
function App() {

    function handleClick() {
        console.log("Button clicked");
    }

    return (
        <button onClick={handleClick}>
            Click Me
        </button>
    );
}
```

Part 20 — State

Now we reach one of the most important React concepts.

Imagine:

```
Courses: 3

[Add Course]
```

We click the button.

It should become:

```
Courses: 4
```

In normal JavaScript:

```
let courses = 3;

courses++;

document.querySelector("#courses").textContent = courses;
```

We manually update the DOM.

React gives us state.

```
import { useState } from "react";

function App() {
    const [courses, setCourses] = useState(3);

    function addCourse() {
        setCourses(courses + 1);
    }

    return (
        <div>
```

```
            <h1>Courses: {courses}</h1>

            <button onClick={addCourse}>
                Add Course
            </button>
        </div>
    );
}
```

The important concept:

```
State changes
      ↓
React re-renders
      ↓
UI reflects new state
```

Part 21 — Conditional Rendering

Suppose our dashboard has authentication.

```
const isLoggedIn = true;
```

We can render:

```
{isLoggedIn ? (
    <Dashboard />
) : (
    <Login />
)}
```

Or:

```
{isAdmin && <AdminPanel />}
```

This lets UI respond to application data.

Part 22 — Forms

Build a simple student registration form.

```
function StudentForm() {
    return (
        <form>
            <input
                type="text"
                placeholder="Student name"
            />

            <input
```

```
                type="email"
                placeholder="Email"
            />

            <button>
                Add Student
            </button>
        </form>
    );
}
```

Then introduce state:

```
import { useState } from "react";

function StudentForm() {
    const [name, setName] = useState("");

    return (
        <form>
            <input
                value={name}
                onChange={(event) =>
                    setName(event.target.value)
                }
            />

            <p>Student: {name}</p>
        </form>
    );
}
```

Now students can see the relationship:

```
Input
 ↓
Event
 ↓
State changes
 ↓
Component re-renders
 ↓
UI updates
```

Part 23 — Parent and Child Communication

Suppose:

```
App
│
└── StudentCard
```

The parent sends information down:

```
App
 ↓
props
 ↓
StudentCard
```

For example:

```
<StudentCard name="Abebe" />
```

But sometimes the child needs to notify the parent.

We can pass a function.

Parent

```
function App() {

    function handleDelete() {
        console.log("Delete student");
    }

    return (
        <StudentCard
            onDelete={handleDelete}
        />
    );
}
```

Child

```
function StudentCard({ onDelete }) {
    return (
        <div>
            <h2>Abebe</h2>

            <button onClick={onDelete}>
                Delete
            </button>
        </div>
    );
}
```

Conceptually:

```
Parent
  ↓ props
Child

Child
  ↑ callback
Parent
```

This is fundamental to component architecture.

Part 24 — `useEffect`

Now we introduce external operations.

For example:

Load students from an API when the page opens.

```
import { useEffect, useState } from "react";

function Students() {
    const [students, setStudents] = useState([]);

    useEffect(() => {
        fetch("/api/students")
            .then(response => response.json())
            .then(data => {
                setStudents(data);
            });
    }, []);

    return (
        <div>
            {students.map(student => (
                <p key={student.id}>
                    {student.name}
                </p>
            ))}
        </div>
    );
}
```

The flow becomes:

```
Component loads
      ↓
useEffect runs
      ↓
API request
      ↓
Data arrives
      ↓
setStudents()
      ↓
React re-renders
      ↓
Students appear
```

Part 25 — API Integration

Now connect React to a real backend.

The architecture becomes:

```
React
  ↓
fetch()
  ↓
Backend API
  ↓
Database
```

For example:

```
fetch("http://localhost:5000/api/students");
```

For creating a student:

```
fetch("http://localhost:5000/api/students", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        name: "Abebe",
        course: "React"
    })
});
```

Students can now connect what they already learned about APIs with React.

Part 26 — CRUD Application

Our project now becomes a real application.

Students implement:

Create

```
Add Student
```

Read

```
Student List
```

Update

```
Edit Student
```

Delete

```
Delete Student
```

The architecture:

```
React
 ├── Components
 │
 ├── State
 │
 ├── Forms
 │
 └── API Services
        ↓
     Backend
        ↓
    Database
```

Part 27 — React Router

Once our application has multiple screens:

```
/dashboard
/students
/courses
/settings
/profile
```

we need routing.

Conceptually:

```
URL
 ↓
Router
 ↓
Component
```

For example:

```
<Route
    path="/students"
    element={<Students />}
/>
```

Then:

```
/students
      ↓
Students component
```

Part 28 — Growing the Architecture

At first:

```
src/
├── App.jsx
└── main.jsx
```

That's fine.

But as our application grows:

```
src/
│
├── components/
│   ├── Navbar/
│   ├── Sidebar/
│   ├── StudentCard/
│   └── Button/
│
├── pages/
│   ├── Dashboard/
│   ├── Students/
│   ├── Courses/
│   └── Settings/
│
├── services/
│   └── studentService.js
│
├── hooks/
│
├── utils/
│
├── App.jsx
└── main.jsx
```

The goal is not to create folders because "React developers do that."

The goal is:

Put related responsibilities together and make the application easier to understand.

Part 29 — Reusable Components

Students should now identify repeated UI.

For example:

```
Button
Card
Modal
Input
Navbar
Sidebar
Loading
ErrorMessage
```

Instead of:

```
<button className="blue-button">
    Add Student
</button>
```

everywhere, we could create:

```
function Button({ children, onClick }) {
    return (
        <button onClick={onClick}>
            {children}
        </button>
    );
}
```

Then:

```
<Button onClick={handleAdd}>
    Add Student
</Button>
```

Part 30 — Custom Hooks

Suppose several components need student data.

Instead of repeating:

```
useState()
useEffect()
fetch()
```

we can create reusable logic.

```
function useStudents() {
    const [students, setStudents] = useState([]);

    useEffect(() => {
        fetch("/api/students")
            .then(response => response.json())
            .then(data => setStudents(data));
    }, []);

    return students;
}
```

Then:

```
function StudentList() {
    const students = useStudents();

    return (
        <div>
            {students.map(student => (
```

```
                <p key={student.id}>
                    {student.name}
                </p>
            ))}
        </div>
    );
}
```

Now the UI component doesn't need to know all the details of fetching students.

Part 31 — Context

Eventually students will encounter information needed by many components:

```
Logged-in user
Theme
Language
Authentication
```

Instead of passing props through many layers:

```
App
 ↓
Dashboard
 ↓
Sidebar
 ↓
Profile
```

we can introduce Context.

This comes later because students first need to understand normal props.

Final Practical Project

The entire course should culminate in:

Student Management System

Students build it incrementally.

Stage 1

Static interface.

```
Dashboard
```

```
Students
Courses
```

Stage 2

Componentize the UI.

```
Navbar
Sidebar
StudentCard
CourseCard
```

Stage 3

Add data.

```
students.map(...)
courses.map(...)
```

Stage 4

Add state.

```
Add Student
Delete Student
Search
Filter
```

Stage 5

Add forms.

```
Create Student
Edit Student
```

Stage 6

Connect API.

```
React
 ↓
Express/Node API
 ↓
Database
```

Stage 7

Add routing.

```
/dashboard
/students
/courses
/settings
```

Stage 8

Refactor architecture.

```
components/
pages/
services/
hooks/
utils/
```

Stage 9

Add authentication.

```
Login
 ↓
Authentication
 ↓
Protected Dashboard
```

Stage 10

Deploy.

```
Development
      ↓
Production Build
      ↓
Deployment
      ↓
Live Application
```

Complete Course Roadmap

Module	Topic	Practical Work
01	Why React?	Analyze a large vanilla JS app
02	UI Reusability	Build reusable JS UI functions
03	ES Modules	Split application into modules
04	npm	Install and use packages
05	`package.json`	Understand dependencies/scripts
06	Vite	Create modern frontend project
07	React Introduction	First React application
08	JSX	Build interfaces with JSX
09	Components	Build dashboard components
10	Props	Create reusable StudentCard
11	Lists	Render dynamic data
12	Events	Make components interactive
13	State	Build interactive dashboard

Module	Topic	Practical Work
14	Conditional Rendering	Login/dashboard UI
15	Forms	Student registration
16	Component Communication	Parent/child interaction
17	`useEffect`	Fetch external data
18	API Integration	Connect React to backend
19	CRUD	Full student management
20	React Router	Multi-page application
21	Component Architecture	Refactor project
22	Reusable Components	Build UI library
23	Custom Hooks	Extract reusable logic
24	Context	Manage shared state
25	Authentication	Protected application
26	Production	Build and deploy
27	Final Project	Complete Student Management System